

```
import nltk

nltk.download('brown')
nltk.download('universal_tagset')

from nltk.corpus import brown
from nltk.tag import HiddenMarkovModelTrainer
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.
```

```
sentences = brown.sents()
print("Total sentences:", len(sentences))
```

```
Total sentences: 57340
```

```
tokens = [word.lower() for sent in sentences for word in sent]
print("Total tokens:", len(tokens))
print("Sample tokens:", tokens[:20])
```

```
Total tokens: 1161192
Sample tokens: ['the', 'fulton', 'county', 'grand', 'jury', 'said', 'friday', 'an', 'investigation', 'c
```

```
tagged_sentences = brown.tagged_sents(tagset='universal')
print("Sample tagged sentence:", tagged_sentences[0])
```

```
Sample tagged sentence: [('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'), ('Ju
```

```
trainer = HiddenMarkovModelTrainer()
hmm_model = trainer.train_supervised(tagged_sentences)

print("HMM Model trained successfully!")
```

```
HMM Model trained successfully!
```

```
trans_prob = hmm_model._transitions
print("Sample Transition Probabilities:")
for k in list(trans_prob.keys())[:5]:
    print(k, trans_prob[k])
```

```
Sample Transition Probabilities:
DET <MLEProbDist based on 137001 samples>
NOUN <MLEProbDist based on 274644 samples>
ADJ <MLEProbDist based on 83690 samples>
VERB <MLEProbDist based on 182648 samples>
ADP <MLEProbDist based on 144758 samples>
```

```
emit_prob = hmm_model._outputs
print("Sample Emission Probabilities:")
for k in list(emit_prob.keys())[:5]:
    print(k, emit_prob[k])
```

Sample Emission Probabilities:  
 DET <MLEProbDist based on 137019 samples>  
 NOUN <MLEProbDist based on 275558 samples>  
 ADJ <MLEProbDist based on 83721 samples>  
 VERB <MLEProbDist based on 182750 samples>  
 ADP <MLEProbDist based on 144766 samples>

```
def predict_next_word(sentence):
    words = sentence.lower().split()
    tagged = hmm_model.tag(words)

    last_word, last_tag = tagged[-1]

    suggestions = []
    if last_tag in hmm_model._outputs:
        emission_dist = hmm_model._outputs[last_tag]
        for word in emission_dist.samples():
            prob = emission_dist.prob(word)
            suggestions.append((word, prob))

    suggestions = sorted(suggestions, key=lambda x: x[1], reverse=True)

    return suggestions[:10], tagged
```

```
sentence = "the government"
suggested_words, tagged_sentence = predict_next_word(sentence)

print("Input sentence:", sentence)
print("POS tagged:", tagged_sentence)

print("\nNext word suggestions:")
for w, p in suggested_words:
    print(w, "->", round(p, 6))
```

```
Input sentence: the government
POS tagged: [('the', 'DET'), ('government', 'NOUN')]

Next word suggestions:
time -> 0.005643
man -> 0.004166
Af -> 0.003607
years -> 0.003419
way -> 0.003204
Mr. -> 0.003063
people -> 0.002936
men -> 0.002671
world -> 0.002482
life -> 0.002453
```

```
print("\nCheck grammatical meaning:")
print("Last word POS:", tagged_sentence[-1])
print("Suggested words are mostly same POS category")
```

```
Check grammatical meaning:
Last word POS: ('government', 'NOUN')
Suggested words are mostly same POS category
```

